ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

# Aula 8

# Sumário

- Representação de números inteiros (continuação)
  - Complemento para 2 (continuação)
    - Overflow
- Circuitos aritméticos

# Complemento para 2 adição

- Adição de números representados em complemento para 2
  - A adição de números representados em complemento para 2 é realizada através da simples adição dos dois números incluindo os bits de sinal
  - Exemplos de adição para operandos com 4 bits:

| | | | | | |
|---|---|---|---|---|---|
| 4 | 0 1 0 0 | 4 | 0 1 0 0 | 4 | 0 1 0 0 |
| + 3 | + 0 0 1 1 | + (-3) | + 1 1 0 1 | + (-4) | + 1 1 0 0 |
| 7 | 0 1 1 1 | 1 | 0 0 0 1 | 0 | 0 0 0 0 |
| 3 | 0 0 1 1 | 3 | 0 0 1 1 | 3 | 0 0 1 1 |
| + 4 | + 0 1 0 0 | + (-4) | + 1 1 0 0 | + (-3) | + 1 1 0 1 |
| 7 | 0 1 1 1 | -1 | 1 1 1 1 | 0 | 0 0 0 0 |

bit de sinal

# Complemento para 2 subtração

- Subtração de números representados em complemento para 2
  - A subtração de números representados em complemento para 2 é realizada através da adição do diminuendo com o simétrico do diminuidor
  - Exemplos de subtração para operandos com 4 bits:

| | | | | |
|---|---|---|---|---|
| 4 | 0 1 0 0 | | 4 | 0 1 0 0 |
| - 3 | - 0 0 1 1 | = | + (-3) | + 1 1 0 1 |
| 1 | 0 0 0 1 | | 1 | 0 0 0 1 |

bit de sinal

| | | | | |
|---|---|---|---|---|
| 3 | 0 0 1 1 | | 3 | 0 0 1 1 |
| - 4 | - 0 1 0 0 | = | + (-4) | + 1 1 0 0 |
| -1 | 0 1 1 1 | | -1 | 1 1 1 1 |

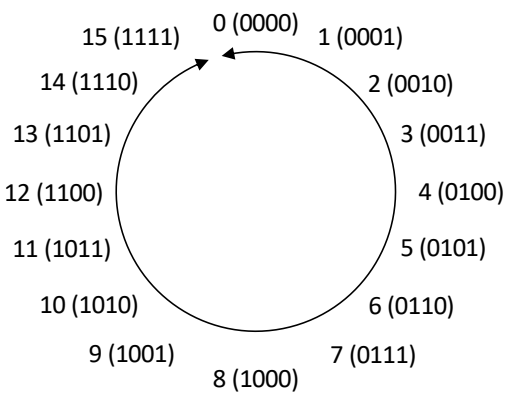
# Capacidade de representação

- Como não é possível aumentar a capacidade de representação nos sistemas digitais, é necessário detetar as situações em que existe *overflow* (situação em que a capacidade de representação é excedida)
- Quando os números são naturais, o *Carry* (para a adição) ou o *Borrow* (para a subtracção) mais significativos assinalam se a capacidade de representação foi excedida (resultado fora de $[0, 2^n - 1]$)
- Para os números inteiros relativos representados em complemento para 2 deteta-se o *overflow* (resultado fora de $[-2^{n-1} - 1, 2^{n-1} - 1]$) usando outras condições lógicas

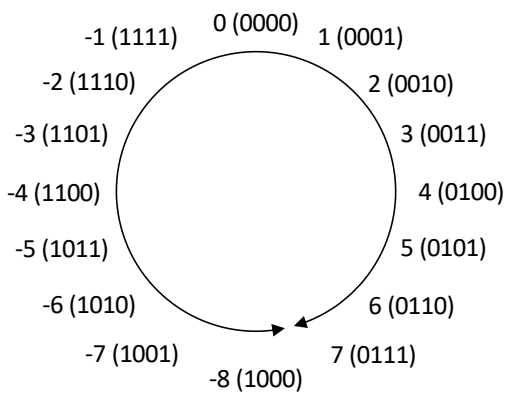
# Overflow

- Capacidade de representação
  - Exemplo para números representados com 4 bits

Naturais



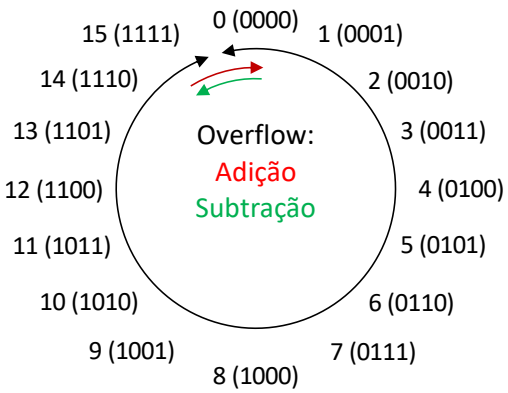

Relativos



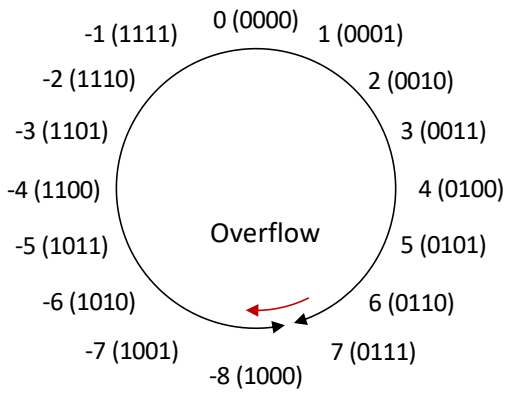
# Overflow

- Capacidade de representação
  - Exemplo para números representados com 4 bits

Naturais
[0, 15]

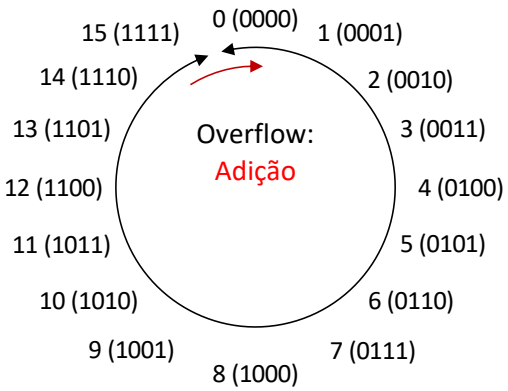



Relativos
[-8, 7]



# Overflow para números naturais

- Deteção de overflow - Adição
  - Exemplo para números naturais com 4 bits




| | | | | | | |
|---|---|---|---|---|---|---|
| | 0 | 0 | 1 | 1 | | 3 |
| + | 1 | 1 | 0 | 0 | + | 12 |
| (0) | 1 | 1 | 1 | 1 | | 15 |

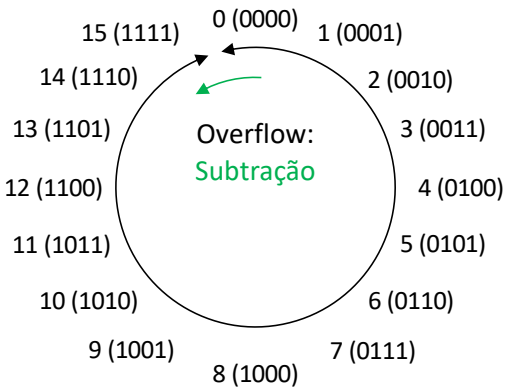
Contido no domínio [0, 15]
CarryOut = 0

| | | | | | | |
|---|---|---|---|---|---|---|
| | 0 | 0 | 1 | 1 | | 3 |
| + | 1 | 1 | 0 | 1 | + | 13 |
| (1) | 0 | 0 | 0 | 0 | (??) | 0 |

Excedeu o domínio [0, 15]
CarryOut = 1

# Overflow para números naturais

- Deteção de overflow – Subtração
  - Exemplo para números naturais com 4 bits




| | | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 0 | 1 | 0 | | 10 |
| - | 0 | 0 | 1 | 1 | - | 3 |
| (0) | 0 | 1 | 1 | 1 | | 7 |

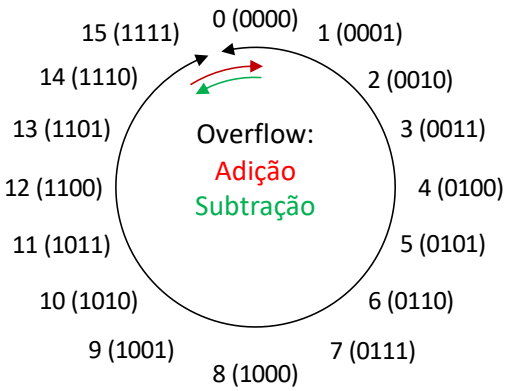
Contido no domínio [0, 15]
BorrowOut = 0

| | | | | | | |
|---|---|---|---|---|---|---|
| | 0 | 0 | 1 | 1 | | 3 |
| - | 0 | 1 | 0 | 1 | - | 5 |
| (1) | 1 | 1 | 1 | 0 | (??) | 14 |

Excedeu o domínio [0, 15]
BorrowOut = 1

# Overflow para números naturais

- Deteção de overflow – Subtração (realizada através de adição)
  - $X - Y = X + (-Y) = X + (2^N - Y) = X + Y_{(C'2)}$ $\qquad$ $\text{BorrowOut} = \overline{\text{CarryOut}}$
  - Exemplo para números naturais com 4 bits




$10 - 3 = 10 + (2^4 - 3) = 10 + 13$

$$\begin{array}{rrrrr} & 1 & 0 & 1 & 0 \\ + & 1 & 1 & 0 & 1 \\ \hline (1) & 0 & 1 & 1 & 1 \end{array} \qquad \begin{array}{rr} & 10 \\ + & 13 \\ \hline (23 \bmod 16) & 7 \end{array}$$

Contido no domínio [0, 15]
CarryOut = 1 => BorrowOut = 0

$3 - 5 = 3 + (2^4 - 5) = 3 + 11$

$$\begin{array}{rrrrr} & 0 & 0 & 1 & 1 \\ + & 1 & 0 & 1 & 1 \\ \hline (0) & 1 & 1 & 1 & 0 \end{array} \qquad \begin{array}{rr} & 3 \\ + & 11 \\ \hline & 14 \end{array}$$

Excedeu o domínio [0, 15]
CarryOut = 0 => BorrowOut = 1

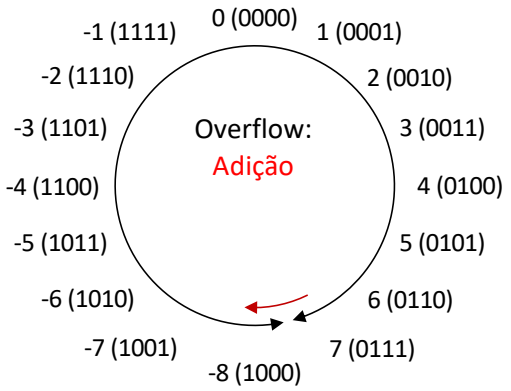
# Overflow para números em complemento para 2

- O overflow para números em complemento para 2 pode ocorrer se forem somados:
  - dois operandos positivos

    ou
  - dois operandos negativos
- A condiçao de *overflow (OV)* pode ser detectada por inspecção dos sinais dos operandos e do resultado:
  - operandos positivos e resultado negativo => overflow
  - operandos negativos e resultado positivo => overflow
  - $OV = A_{n-1}.B_{n-1}.\overline{S_{n-1}} + \overline{A_{n-1}}.\overline{B_{n-1}}.S_{n-1}$

# Overflow para números relativos (complemento para 2)

- Deteção de overflow
  - Exemplo para números em complemento para 2 com 4 bits




| | | |
|---|---|---|
| | 0 0 1 1 | 3 |
| + | 0 0 1 0 | + 2 |
| | 0 1 0 1 | 5 |

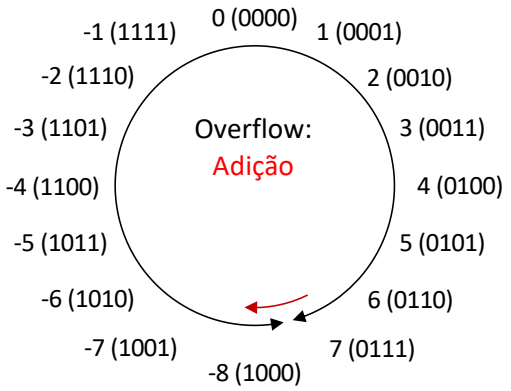
Contido no domínio [-8, 7]
Overflow = 0

| | | |
|---|---|---|
| | 0 0 1 1 | 3 |
| + | 0 1 1 0 | + 6 |
| | 1 0 0 1 | (??) -7 |

Excedeu o domínio [-8, 7]
Overflow = 1

# Overflow para números relativos (complemento para 2)

- Deteção de overflow
  - Exemplo para números em complemento para 2 com 4 bits
  - Subtração é realizada à custa de somas: $X - Y = X + (-Y)$




6 – 5 = 6 + (-5)

| | | | | | |
|---|---|---|---|---|---|
| | 0 | 1 | 1 | 0 | 6 |
| + | 1 | 0 | 1 | 1 | + (-5) |
| | 0 | 0 | 0 | 1 | 1 |

Contido no domínio [-8, 7]
Overflow = 0

– 5 – 6 = – 5 + (– 6)

| | | | | | |
|---|---|---|---|---|---|
| | 1 | 0 | 1 | 1 | -5 |
| + | 1 | 0 | 1 | 0 | + -6 |
| | 0 | 1 | 0 | 1 | (??) 5 |

Excedeu o domínio [-8, 7]
Overflow = 1

# Complemento para 2
# Overflow

- Exemplos

Soma de dois números positivos

$$\begin{array}{rl} 0\ 1\ 1\ 0 & +6_{(10)} \\ +\ 0\ 0\ 1\ 1 & +3_{(10)} \\ \hline \end{array}$$

# Complemento para 2
# Overflow

- Exemplos

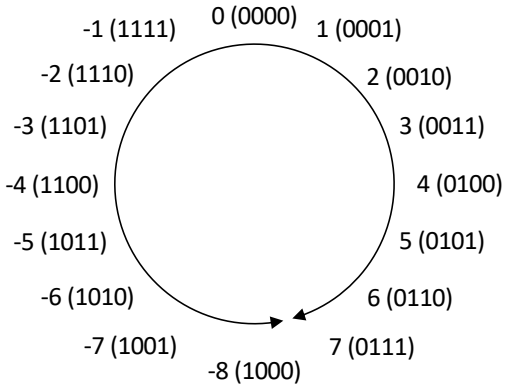
Soma de dois números positivos

$$\begin{array}{rl} 0\ 1\ 1\ 0 & +6_{(10)} \\ +\ 0\ 0\ 1\ 1 & +3_{(10)} \\ \hline 1\ 0\ 0\ 1 & \end{array}$$

# Complemento para 2
# Overflow

- Exemplos

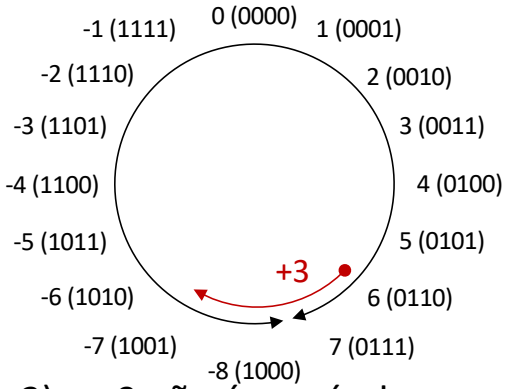
| | |
|---|---|
| 0 1 1 0 | $+6_{(10)}$ |
| + 0 0 1 1 | $+3_{(10)}$ |
| 1 0 0 1 | $-7_{(10)}$ ???? |



# Complemento para 2
# Overflow

- Exemplos

| | |
|---|---|
| 0 1 1 0 | $+6_{(10)}$ |
| + 0 0 1 1 | $+3_{(10)}$ |
| 1 0 0 1 | $-7_{(10)}$ ???? |




- Ocorreu *overflow* porque o valor (6+3) = +9 não é possível ser representado em código de complemento para 2 com 4 bits
  - Domínio complemento para 2 com 4 bits: [-8, 7]

# Complemento para 2
# Overflow

- Exemplos

Soma de dois números negativos

$$\begin{array}{rl} 1\ 1\ 0\ 0 & -4_{(10)} \\ +\ 1\ 0\ 1\ 0 & -6_{(10)} \\ \hline \end{array}$$

# Complemento para 2
# Overflow

- Exemplos

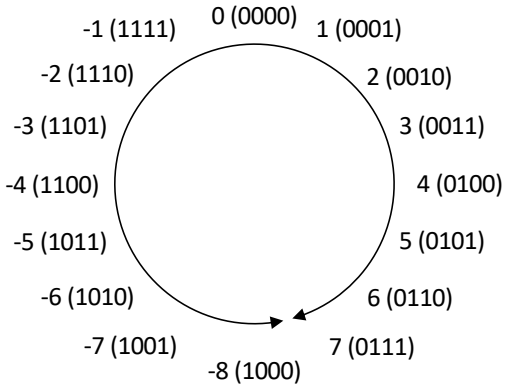
Soma de dois números negativos

$$\begin{array}{rl} 1\ 1\ 0\ 0 & -4_{(10)} \\ +\ 1\ 0\ 1\ 0 & -6_{(10)} \\ \hline 0\ 1\ 1\ 0 & \end{array}$$

# Complemento para 2
# Overflow

- Exemplos

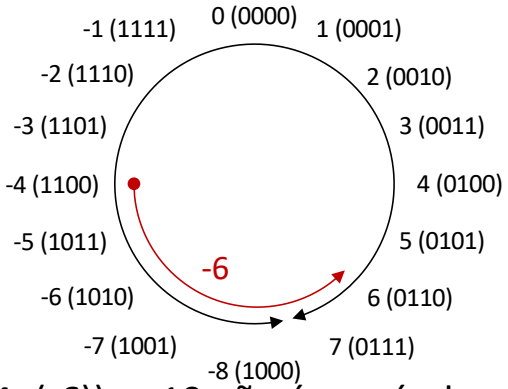
| | |
|---|---|
| 1 1 0 0 | $-4_{(10)}$ |
| + 1 0 1 0 | $-6_{(10)}$ |
| 0 1 1 0 | $+6_{(10)}$ ???? |



# Complemento para 2
# Overflow

- Exemplos

| | |
|---|---|
| 1 1 0 0 | $-4_{(10)}$ |
| + 1 0 1 0 | $-6_{(10)}$ |
| 0 1 1 0 | $+6_{(10)}$ ???? |




- Ocorreu *overflow* porque o valor (-4+(-6)) = -10 não é possível ser representado em código de complemento para 2 com 4 bits
  - Domínio complemento para 2 com 4 bits: [-8, 7]

ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

# Circuitos aritméticos

# Soma binária

- Fazendo a correspondência entre dígitos na base binária e valores lógicos booleanos, podemos extrair as seguintes funções booleanas para $S_n$ e $C_{n+1}$:

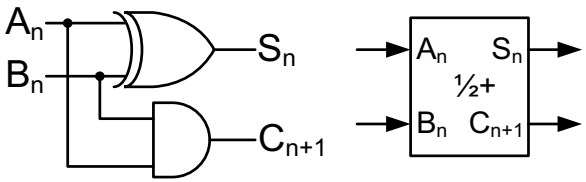
| $A_n$ | $B_n$ | $S_n$ | $C_{n+1}$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

- $S_n = A_n.\overline{B_n} + \overline{A_n}.B_n = A_n \oplus B_n$
- $C_{n+1} = A_n.B_n$

# Semi-somador (*half-adder*)

- O circuito semi-somador (*half-adder*) soma 2 bits de entrada (sem transporte/*Carry*) e produz 1 bit de soma e 1 bit de transporte

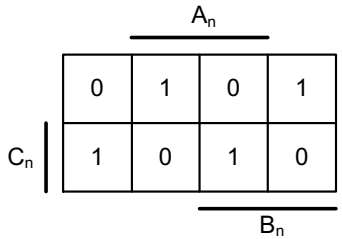
- $S_n = A_n \oplus B_n$
- $C_{n+1} = A_n . B_n$



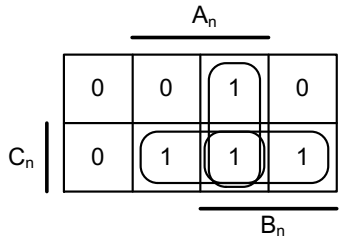
# Somador completo (*full-adder*)

- O circuito somador completo (*full-adder*) soma 3 bits de entrada (incluindo o transporte/*Carry* anterior) e produz 1 bit de soma e 1 bit de transporte

| $A_n$ | $B_n$ | $C_n$ | $S_n$ | $C_{n+1}$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |



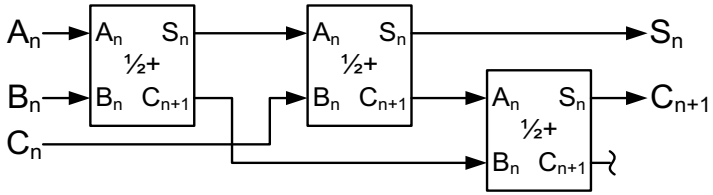

$$S_n = A_n \oplus B_n \oplus C_n$$



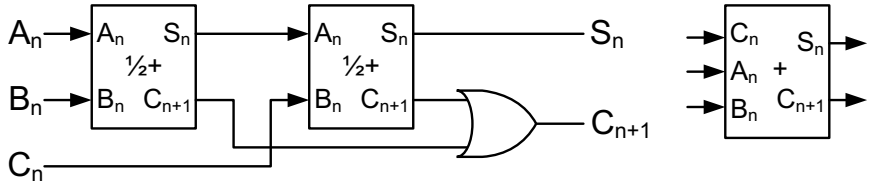

$$C_{n+1} = A_n.B_n + A_n.C_n + B_n.C_n$$

# Somador completo (full-adder)

- Podemos construir um somador completo utilizando como bloco base o semi-somador, visto que a soma é associativa: A+B+C = (A+B) + C

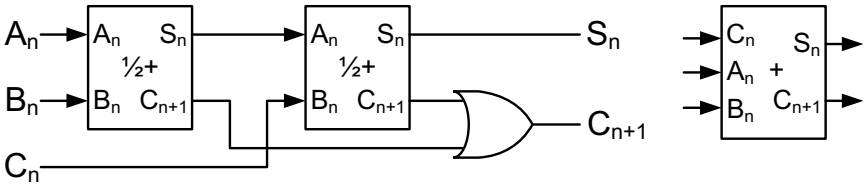


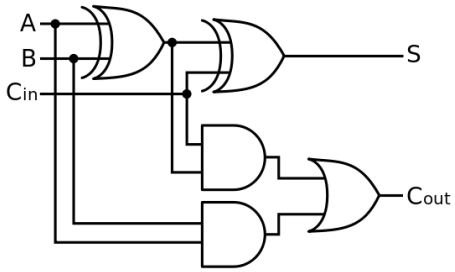

- Visto que o terceiro somador não produz transporte (*Carry*), poderá ser substituído por um XOR ou por um OR



# Somador completo (full-adder)

- O circuito somador completo (*full-adder*) soma 3 bits de entrada (incluindo o transporte/*Carry* anterior) e produz 1 bit da soma e 1 bit de transporte

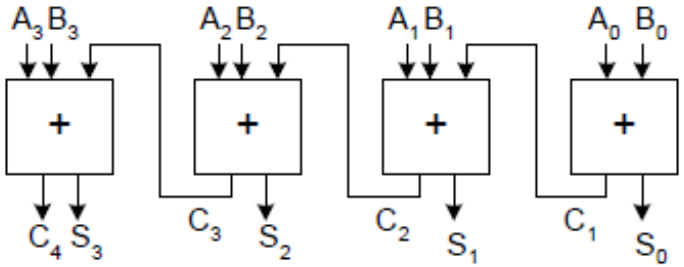



- $S_n = A_n \oplus B_n \oplus C_n$
- $C_{n+1} = A_n \,.\, B_n + (A_n \oplus B_n).\, C_n = A_n.\, B_n + A_n.\, C_n + B_n.\, C_n$

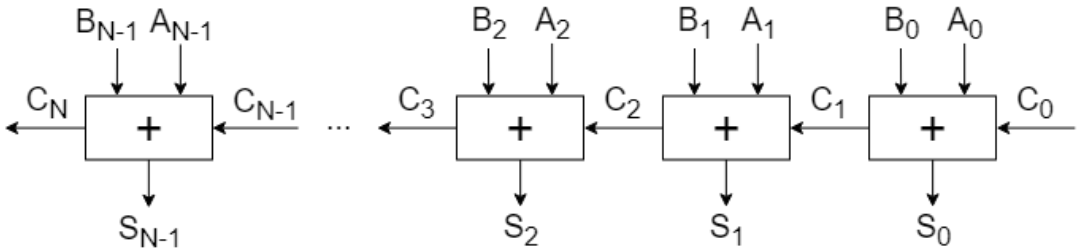
# Somador de n bits

- O somador de *n* bits soma dois números (A e B) com *n* bits cada
- As saídas são:
  - Soma ($S_3..S_0$), igualmente com *n* bits
  - Carry ($C_n$)
- Solução iterativa:
  - Utiliza módulos replicáveis (*full-adders*)



# Somador em cascata (*ripple carry adder*)

- Utiliza uma solução iterativa
- Desvantagem:
  - Tempo para o cálculo (i.e. o tempo de propagação) é grande
    - Deve-se à propagação do carry pelos módulos somadores (desde o 1º bit até ao último)
    - No pior caso é N * $t_{pfa}$ ($t_{pfa}$ = tempo propagação do *full adder*, 3 portas lógicas)
- É o somador que requer o menor número de portas lógicas



# Outros somadores

- Existem várias alternativas ao somador em cascata, com diversos compromissos entre área e velocidade:
  - *Carry select adder*
  - *Carry look ahead adder*
  - …
- A alternativa mais rápida, mas com maior número de portas lógicas, é o *Carry look ahead*

# Circuito subtrator
# Representação de números inteiros

- Considerando que representar em complemento para 2 é o mesmo que representar em complemento para 1 e adicionar 1, i.e:
  - $\boxed{(2^n) - N} = \boxed{[(2^n - 1) - N]} + 1$

complemento para 2 = (complemento para 1) + 1

(em base 2, o complemento para 1 equivale a inverter todos os bits)

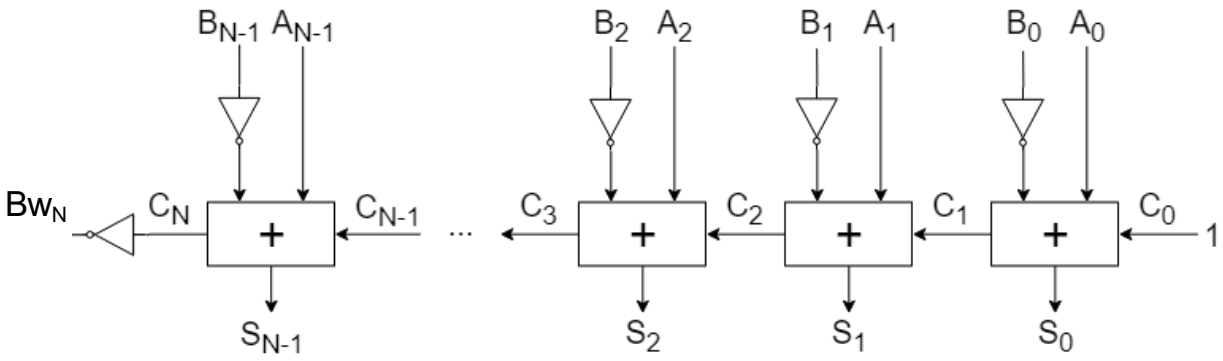
- A subtracção usando a representação em complemento para 2 pode ser obtida usando um somador onde o operando a subtrair tem os seus bits invertidos (complemento para 1) e a soma de 1 é obtida colocando o *Carry* de entrada a 1

# Circuito subtractor

- Usando o circuito somador anteriormente apresentado, podemos alterá-lo para realizar a função de subtracção de números binários usando representação em complemento para 2

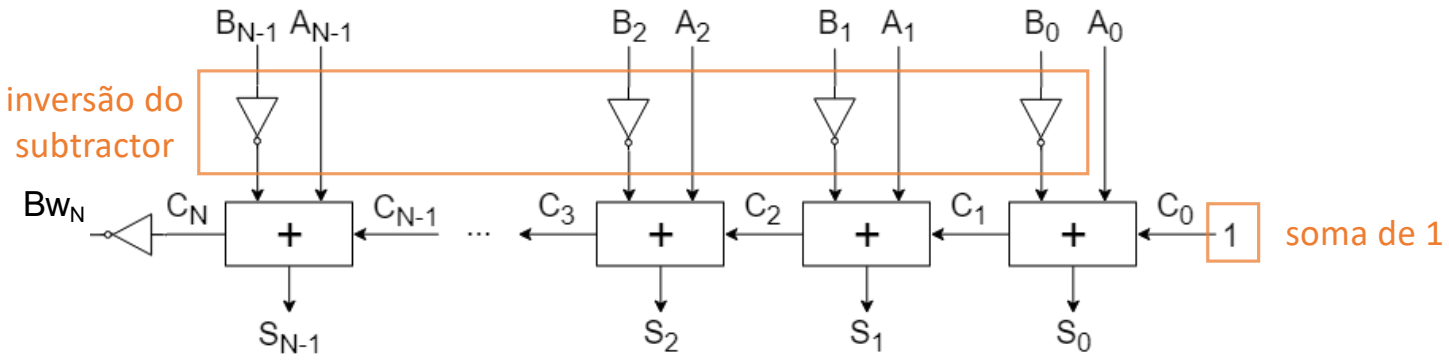
# Circuito subtractor

- Usando o circuito somador anteriormente apresentado, podemos alterá-lo para realizar a função de subtracção de números binários usando representação em complemento para 2



# Circuito subtractor

- Usando o circuito somador anteriormente apresentado, podemos alterá-lo para realizar a função de subtracção de números binários usando representação em complemento para 2



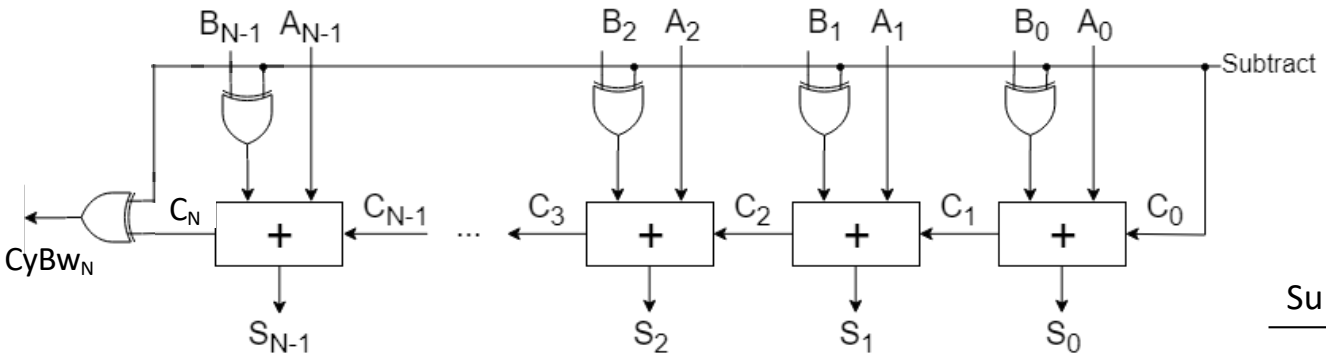
# Circuito somador/subtractor

- Utilizando um inversor controlado (XOR), um único circuito pode ser usado para realizar as operações de soma e subtracção de números binários usando representação em complemento para 2

| Subtract | Função |
|---|---|
| 0 | Adição |
| 1 | Subtracção |

# Circuito somador/subtractor

- Utilizando um inversor controlado (XOR), um único circuito pode ser usado para realizar as operações de soma e subtracção de números binários usando representação em complemento para 2




| Subtract | Função |
|---|---|
| 0 | Adição |
| 1 | Subtracção |

# Overflow

- Para os números naturais deteta-se o *overflow* quando:
  - na operação de soma, o *Carry* mais significativo tiver o valor 1
  - na operação de subtração, o *Borrow* mais significativo tiver o valor 1

- Para os números inteiros relativos representados em complemento para 2 deteta-se o *overflow* se a operação de soma tiver:
  - operandos positivos e resultado negativo

  ou
  - operandos negativos e resultado positivo

# Overflow em complemento para 2

- As condições lógicas para detecção de *overflow* (OV) na soma de 2 operandos com *n* bits é dada pela expressão:
  - $OV = A_{n-1}.B_{n-1}.\overline{S_{n-1}} + \overline{A_{n-1}}.\overline{B_{n-1}}.S_{n-1}$
- Dada a função lógica do circuito somador completo, podemos deduzir que a expressão anterior é equivalente a:
  - $OV = A_{n-1}.B_{n-1}.\overline{C_{n-1}} + \overline{A_{n-1}}.\overline{B_{n-1}}.C_{n-1}$
    - Para $A_{n-1}$=1 e $B_{n-1}$=1, $S_{n-1}$=0 sse $C_{n-1}$=0; Para $A_{n-1}$=0 e $B_{n-1}$=0, $S_{n-1}$=1 sse $C_{n-1}$=1
  - $OV = C_n.\overline{C_{n-1}} + \overline{C_n}.C_{n-1}$
    - Quando $A_{n-1}$=1 e $B_{n-1}$=1, $C_n$=1; Quando $A_{n-1}$=0 e $B_{n-1}$=0, $C_n$=0
  - $OV = C_n \oplus C_{n-1}$

| | | n | n-1 | n-2 |
|---|---|---|---|---|
| carry | | 1 | 0 | - |
| A | | | 1 | - |
| B | + | | 1 | - |
| S | | | 0 | - |

| | | n | n-1 | n-2 |
|---|---|---|---|---|
| carry | | 0 | 1 | - |
| A | | | 0 | - |
| B | + | | 0 | - |
| S | | | 1 | - |

# Diferença entre Carry e Overflow

| **Representação** | **CarryBorrow**<br>**$CyBw_n = 1$** | **OV = 1** |
|---|---|---|
| Naturais (sem sinal) | Capacidade de representação excedida | Sem significado |
| Complemento para 2 (com sinal) | Sem significado | Capacidade de representação excedida |

# Temas abordados

**Sumário**

- Representação de números inteiros (continuação)
  - Complemento para 2 (continuação)
    - Overflow
- Circuitos aritméticos

**Próxima aula**

- Laboratório C